Notes on the Stanford GraphBase

Adam Martinez*
University of Life

1 Project code organization

The main project has apparently been improved by a contributor other than DEK, including patches for what seems like compatibility with post–ANSI C code, as well as better pointer handling practices. The changes beyond those in specific folders like `AMIGA` or `MSVC` (i.e. those included as patches in the project's root directory,) have not yet been inspected. Implementing the changes included in those files will not be done, as apparently the core logic was sound enough to even be implemented in both the Boost Graph Library, and in a subproject of it.

As per the project's `README`, all logic is implemented in terms of the *kernel routines*, so called because they implement the graph DSs in use as well as some routines for efficiently handling both random number generation and linked list traversal (maybe because adjacency lists are implemented in terms of linked lists?)

The actual logic for graph generation is stored in all `gb_*` files other than (1) `flip`, (2) `graph`, (3) `io`, and, (4) `sort`, except for (a) `dijk`, (b) `save`, and, (c) `types`. I believe the main approach to this should be an inspection of the kernel routines, followed by an in–depth study of the generative routines. Hopefully, this can yield some conclusions as to whether a generic interface over the graph kernel routines can be implemented, such that a different "backend" can be interchangebly used with the same generative routines.

1.1 Kernel code files

1.1.1 Random number generation (`gb_flip.w`)

The interface to the code is fairly simple, and is apparently based off of a publication of the name *Seminumerical Algorithms*. I may require this article if the engine proves too complex for me to implement without further external assitance.

In and of itself, this part of the program offers a function with which to initialize the random number generator, and a macro with which to produce a random number. Both of these are very much transparent in the way the perform their internal operations, as the initial routine expects an explicit seed with which (for now, I believe) the program "picks" a point in its deterministic sequence to start

*`staying@never.land`

off producing values. Beyond this, the macro to be called makes explicit the fact that the generated numbers follow as part of, upon initialization, a predetermined series.

According to the file, the period of the numbers is of $2^{85} - 2^{30} = 2^{55}$. According to Skiena's book, the cycling of numbers that rely on 2^{32} calls of a linear congruential engine is worrying. Whether the algorithm used in this file is a linear congruential engine, and whether 2^{55} calls may be performed by today's computers in little more than 2^{32} calls is something I am not aware of.

Further inspection of the file reveals that this seems very much like an instance of a linear congruential engine, where the value of a random number n is determined as the function R_n, such that

$$R_n = (R_{n-55} - R_{n-24}) \bmod m, \tag{1.1}$$

where m is even and $R_0, R_1, \cdots, R_{54}$ is an arithmetic series containing both even and odd numbers.

This looks a lot like the computation resolved in the example in [1, Sec. 16.7, p. 487]. It computes the value of the nth random number from some other $n - 55$th and $n - 24$th random numbers. This algorithm is also noted to consider m as taking on the largest value with which to bound the number the recurrence relation in the modulo's lhs resolves to, by taking on the 2^{31} full range of unsigned integer values.

In terms of the effectiveness of such a random number generator, DEK notes that the chosen offsets, namely 24 and 55, should prove to be good enough for "most" applications. This likely doesn't cover cryptographic applications, but neither do I belive the Stanford GraphBase to require of fine–grained pseudorandom number generation. The point being, **the period of this generator should be at least $2^{55} - 1$.**

The C code generated after running `ctangle` is full of `#line` directives I'm going to ignore for the sake of sanity. Maybe there's more information on this in [2], [3]. The code then seems fairly straightforward.

The "public" interface in the header file exposes both the macro for advancing the random number sequence, as well as three more routines.

`gb_flip_cycle()`
: Yet unbeknownst to me.

`gb_init_rand()`
: To initialize the random number engine with a given seed.

`gb_unif_rand()`
: Yet unbeknownst to me.

Beyond this, the state of the whole shebang is kept through two globals; (1) an array `A` holding $n + 1$, where $n = 55$ elements, and, (2) a pointer to that array `gb_fptr`. The array is set to hold one more than the largest offset used in Equation (1.1), such that the first value is used as a sentinel (more on this once I get to know the code better.) The pointer doesn't exist for any purpose other than allowing the user to run the macro exposed in the header file (which otherwise would not have any means of reaching for the array holding the core state.)

Of note is that the documentation speaks of a limitation in the `gb_next_rand` macro: There seems to be a test, referred to as the *birthday spacings test*, that fails to prove this to be a decent enough random number generator. The solution proposed by DEK is to modify the definition of the macro such that instead of performing the following computation,

```
#define gb_next_rand() (*gb_fptr >= 0 ? *gb_fptr-- : gb_flip_cycle())
```

it performs two cycle flipping computations in a row before continuing execution.

```
#define gb_next_rand() (*gb_fptr >= 0 ? *gb_fptr-- : (gb_flip_cycle(),  \
                                                       gb_flip_cycle()))
```

The modification would exploit the comma operator in C to allow for two consecutive runs of the routine prior to continuing with the flow of execution at the macro invocation site; This is possible thanks to the fact C evaluates as `void` the lhs of the comma operator and assures that the rhs will only run after the lhs has finished execution, such that the `gb_flip_cycle()` returns the last element of the stateful array after having run 110 steps of the recurrence defined in Equation (1.1). For future reference, I belive it is useful that we inspect the operations performed in the macro.

First, it considers whether the current value pointed to by `gb_fptr` is negative. This holds only for the sentinel value in the array the pointer it is aliasing, and thus serves as an indication that it's time to run `gb_flip_cycle()` (though the workings of this are, yet again, unbeknownst to me at this point.) If the value yielded happens to still be within the "acceptable" range, then the underlying value is dereferenced again prior to performing pointer arithmetic by subtracting from `gb_fptr`; Note how the decrement operator is used in its postfix form, such that only upon returning the value does the pointer's address recede back by one position in the array.

The `gb_flip_cyle()` routine is said to perform 55 iterations of Equation (1.1), aiming for these to be as high–speed as possible by requesting a register storage class specifier of the pointers in use. This function's body, though, is quite the sight for sore eyes; It keeps two pointers to the array holding the n random numbers (never acting on the sentinel value at index `1`,) and performs *pointer address* comparisons to consider whether the pointer at the end of each loop iteration has hit the address of the last element in the array. The problem here is that the exit condition of the loops depends on whether

the pointer involved in each one, respectively, has an address that is now "beyond" the address range of the array (i.e. has an address that is numerically larger than that of the last element of the array.) Technically, one can trust that C stack–based arrays will allocate contiguous memory and thus an address that is numerically larger than the address of the last element in the array would be outside the safe range in which to dereference the pointer, so the check is certainly not incorrect in its logic. But this is borderline unsafe in Rust.

Then it proceeds to "reset" the `gb_fptr` pointer by making it alias element at index `54` of the stateful array. I belive it resets it to the element right before the last and not to the element before the last proper because the routine itself returns the last element in the stateful array. And then because this function is really only used inside the `gb_next_rand` macro, it's expected to keep a coherent sequence of values, such that so long as we've not hit the sentinel, we return the dereferenced `gb_fptr`, otherwise calling `gb_flip_cycle()` and getting after its call the value at the very end of the array, while resetting back `gb_fptr` for the next call to the macro to start anew.

The `gb_next_rand` macro simply computes the actual formula in Equation (1.1) with discrete values for the terms R_{n-55}, R_{n-24}. This, though, is not a built–in modulo operation with the chosen m (2^{31}), but rather a (possibly) more optimized version using a bit–wise AND that relies on the machine using 2C bit representation for its integer primitive types.

Note that, according to the docs, the `gb_flip_cycle()` routine is to be thought as reflecting the sequence of values, in the sense that they are now considered in reverse order to their initial orderings. Still, a point is made about this not affecting the degree of perceived randomness on the returned sequence throughout calls to the `gb_next_rand` macro and subsequent "flipped cycles" upon hitting the sentinel value in the array with the `gb_fptr`.

The initialization routine `gb_init_rand()` follows a process akin to the one detailed in *Seminumerical Algorithms*, except that apparently the summary that it references bases its generator off of the assumption that only the low–order bits of the initial values (those variables allocated at the start of the routine) are the ones with pseudodeterministic significance. Then for the initial number sequence dispersion, it makes use of coprime numbers 21 and 55 because further increments expressed in terms of a modulo such as $21 \bmod 55$ allow for the iteration–based values in use with the initialization of the seed to be numbers part of the Fibonacci sequence (this is commented to be an alternative method of improvement in TAoCP once the seed value has determined a starting point in the precomputed arithmetic series.) The reason why this is any relevant for the purposes of a random number generator are discussed in *Sorting and Searching*.

The reason why the resulting C programs from running `ctangle` on the CWEB sources make abundant use of the `#line` directive is due to the fact literate programming, as conceived by DEK in WEB, may clip parts of a given routine or general language construct, for the sake of documenting an isolated "region" of it. This in turn forces `ctangle` to parse and require a restructuring that may not be desired when debugging and using compiler–defined symbols when the C source file changes the ordering of such lines to the one expected by a compiler toolchain. This in turn implies DEK expects CWEB programs to be perused in their `.w` forms, and not as standalone C programs, including debugging.

Back to the initialization routine, this process mostly consists of three separate steps: (1) Assigning to each value of the statful array a different "random" value, (2) computing the next set of values that will be assigned to such elements of the array, and, (3) "warming up" the values finally set in the array by calling for 275 steps of the array value–reflecting routine within the cycling function. The reason behind the warmup cycles being run after the example routine in *The Art of Computer Programming* is due to the fact that the least 10 significant bits (the low–order bits we spoke of before) present a fairly predictable pattern no matter which first random number we compute. Of course, the pattern is only obvious when purposefully considering the bits of the numbers, even if small fluctuations may happen between the 9th bit and the 1st bit. The quick cycling (array member reflection) added to the initialization routine for this pattern is meant to quickly disperse values, as otherwise the first few hundred runs would very much follow step no matter the environment execution conditions.

Beyond this, there's nothing else to the generator routines. The only other function present in the public interface of the library is one for computing a uniform, *bounded* distribution of integers. As per S. Skiena, the function R_n already produces such bounded distribution, where the range is denoted as $[0, m)$, so it's quite possible the function presented in DEK's generator is not a linear congruential engine, but can be coerced into the ranges produced by one. The reason why the routine is provided instead of simply bounding the generated number by a modulo operation is attributed to the fact that such operation would yield values smaller than or equal to $^{m}/_{2}$, on $^{2}/_{3}$ of the runs. This function should apparently (though no further elaboration on the reason why is given) try to clamp the genearted value down to the specified range, while not consuming any more than 2 random numbers in the precomputed series (through the `gb_next_rand` macro.)

This seems to work especially well for values of m larger than 2^{16}, where the trend for smaller values is seen more often. Because the random numbers generated within the hot loop of the routine are compared with the largest representable 32–bit *unsigned* integer modulo m, this should technically perform a form of await operation that would evaluate the generated random number prior to itself being returned modulo m. This may sound like more than two runs (and thus more than two macro invocations to `gb_next_rand`) would be required, but of note is that the generated number is itself

not yet bound to a range smaller than the provided m, even if the heuristic used to compute a number anew is based on a value bound within m (more specifically, $2^{31} \bmod m$.) It is only once the computed value yields a (possibly) larger value than those generated under $m/2$ that the routine returns the "clamped" value $n \bmod m$ (I use n instead of the variable name `r` in the source code, for the purpose of generality.)

Pending: getting information on some omitted technicalities on the methods used for both engine initialization and number generation (though these should be found in [4]).

1.1.2 Graph routines (`gb_graph.w`)

The file speaks of safety in terms of undefining a `min` macro that seems to exist in certain system headers, but it uses considerably unsafe practices for pointer arithmetic when performing traversals in a loop.

All declarations until linea 110 of the generated C code are part of the type system used for graph abstractions. It also seems to include a `verbose` and `panic_code` variables to consider for (1) possible additional verbosity in the output of graph routines (akin to current–day `--verbose` CLI options), and, (2) a custom, `errno`–like, global return code living as a static uninitialized variable for the purposes of reporting a wide assortment of failure cases, respectively.

The error codes returned use a sligthly primitive approach based on the same `E*` error codes present in UNIX–based platforms, but really only using generic names for, say, a syntax error, where there may be different variations in the core source, but such subtleties are left to the engineer to find out by perusing the implementation in search of the offset added to the generic error code. As a general rule, DEK allows for the following ranges to delimit the set of values that may be stored in the `panic_code` global.

Code -1

Some memory limit was hit during some prior operation. This is not further explained, and I don't know how would a single error buffer hold an indication of a "previous" error if determining that the error itself took place before some other error already requires having had another error, and thus having had to discard the "previous" error to use this integer buffer for the new error code.

Range 1–9

Some memory limit has been hit, quite possibly due to heap or stack allocation limits, though I doubt the program uses the `alloca` syscall.

Range 10–19 (10 with an additional offset 1–9)

This denotes some error took place at the "start" of one of the data files sourcing the information

that gets parsed onto the graphs. Note that we speak of a *10 with an additional offset* because the next item denotes variations on the errors when the error has taken place at some point in the second middle of the `.dat` file at fault.

Range 11–19 (11 with an additional offset 1–8)

Much in the same way as the above range, it reports some error has taken place at some point in the "lower end" of the file (the second half of it.)

Range 20–29

There was a syntax error while reading a `.dat` file. I assume this error implies the above two ranges relate to memory corruption errors in the I/O routines, and not so much in the file parsing procedure.

Range 30–31

There was an error related to the set of parameters passed to some kernel routine. This range is reserved for fairly "well–behaved" but either way wrong values that may still make some sense in terms of the operation performed by the routine at hand.

Range 40–49

Ibid., except the passed values are completely wrong and it could very well be that the overarching logic involved in the call to the reporting subroutine is also wrong. Basically, you're *stupid* (in the words of DEK himself.)

Range 50–59

The graph parameter is `NULL`, which is no good but especially so in this codebase, where according to the `README`, DEK *assumed* that this symbol was defined universally as `0` and thus is used for expansions where one would naturally expect the number 0 used. **Pay attention to this item, both because in Rust this could very well be a no–op, and because the code could be quite wrong in its use of `NULL`.**

Range 60–89

A parameter is technically correct, but it likely breaks some invariant the routine must upkeep as part of its pre/post–conditions (maybe.)

Range 90–onwards

This is not meant to happen, but in the name of safety (the irony,) DEK still considers these edge cases, and binds values to be interpreted as execution paths ending at an `unreachable!()` macro in Rust.

The docs move on now to the DSs in use for graph abstractions. Because of the presence of multiple types of graphs across the actual library routines in GraphBase, graphs use an overarching structure `Graph`, which itself is accompanied by types for edges and vertices. The edges use an `Arc` type, while the vertices use a `Vertex` type.

Each graph uses an array (though at this point it's unspecified whether it is stack or heap allocated; The base assumption should be that of a contiguous memory allocation) to store `Vertex` structures. Then it uses a linked list (this one must be heap–allocated, for Rust's sake) to keep a record of the edges, such that each element of the linked list is an `Arc` structure.

By the sound of it, this seems to be using an adjacency list, where the array of `Vertex`s possibly has that each of those structures contains a field with a pointer to the linked list of adjacent edges as `Arc`.

Another core tenant of the architectural design is that each one of the `Arc`, `Vertex` and `Graph` types is provided with an assortment of additional fields of a union type `util`. This union contains one of (1) a pointer to another `Graph`, (2) a pointer to another `Vertex`, (3) a pointer to another `Arc`, (4) a string literal as a pre–ANSI C `char *`, or, (5) a 32–bit signed integer. This union is said to be provided in the spirit of allowing flexibility for the algorithms using these set of DSs. **This could quite possibly benefit from a complete refactor in Rust to extend said flexibility far beyond the one achieved by a few union fields in a structure**.

The set of fields in the `Vertex` structure is fairly straightforward. It associates a string literal with the `Vertex` in question, as well as a standard C implementation of a singly linked list for the adjacency list, such that only a pointer to an edge `Arc` is really held per vertex. Then an invariant holds such that if the pointer is non-`NULL`, one may assuredly access the `Arc` and check for the null–ness of a `next` field giving step to the next edge *with respect to the original `Vertex` from which the pointer to `Arc` originated*. Each vertex is also outfit with 6 `util` unions, which are said to be useful for some such information as vertex degrees or additional relationships to other elements of a given graph.

The set of fields in `Arc` structures hold information about the vertex that some other vertex is connected to (implying here that such relationship only really holds true when sourcing an `Arc` from the adjacency list of a `Vertex`, where this latter vertex makes up the other end of the edge,) the next `Arc` connecting the source `Vertex` (in the same context as the above note) in such vertex's adjacency list, and a length associated with the edge (possibly for weighted graphs or otherwise embedded/isomorphic graphs/ needs for graphs.) Unlike `Vertex`, it holds a smaller number of `util` unions, namely 2, for purposes of diversifying use in routines external to GraphBase's kernel functions (this may imply that `Arc`s are expected to hold far less meaning, and thus extensions to the core functionality should focus on `Vertex` structures.)

This design for vertices and edges does force the same set of restrictions on graph representations as basic adjacency lists presented in [5]. A more optimized graph backend set of DSs is definetely a better idea, and will likely motivate the implementation of the Engine API as a Rust trait–based implementation of this and other routines.

The docs move on now to explaining the memory allocation strategy followed to try and make the system efficient. This is quite possibly going to require both considerable refactoring in Rust and benchmarks to test that whatever efficency was attained with DEK's methods continues being *attainable* with Rust methods.

The way memory allocations and memory resources are freed is by means of an abstraction layer over basic memory handling library functions of whatever pre–ANSI C the author used. The entrypoint and main resource handle in all routines involved is the so–called `Area` type(def). This could be said to be modeled after the `allocator` API in the corresponding module of Rust's `std`, only instead of being a complete memory allocator, it serves as a bridge to mediate between the default system allocator and whatever memory allocation requests are performed in the graph routines.

The `Area` type is, at its core, a singleton array of pointers to other `Area`s (to allow for direct dereferencing through array–to–pointer decay without expecting the user to provide the explicit address–of operator in parameters to memory allocation request routines.) When rid of this level of indirection, the underlying structure reveals to be a `char *` (this is from back when `char *`s were used instead of `void *`s because this latter type wasn't valid, "generic" C code,) denoting the address at which this block of "memory area" starts. This is also accompanied by another field `next` pointing to another `Area`. I don't completely understand whether this field acts as a form of linked list connection to the next (or previous?) block of allocated memory. The documentation mentions it's a pointer to the start of the previously allocated block, but also implies that a block is made out of multiple allocations acting as the pointees of differing sources.

Maybe `Area`s are really only an abstraction over a zeroed memory block, that is not itself meant to be used as memory for an object, but rather as pre–allocated resources to make requests on. Maybe.

The basic use of `Area`s follows that the user should consider in which segment of the program the resource handle is stored at, and if it is outside the statically initialized data segment, so either in *bss* or in stack space (I don't think it a good idea to have it in the free store) then it should proceed to call the `init_area` macro to initialize the array to `NULL` (this, for one, is a correct use of the symbol.) Each `Area` represents a fixed–size heap allocation, and each call to the `gb_alloc()` routine with a block size and an `Area` passed as parameters is expected to either (1) allocate on a `NULL` memory

area, (2) extend a non–`NULL` memory area, or, (3) return `NULL` (as a status code, so quite wrongly assuming it expands to `0` on all platforms) if the `Area` cannot allocate any further.

Note the memory allocation limits set on this routine are not given by system limits (reacting to an error to C allocation routines,) but rather by a hardcoded magic number `0xffff00` (i.e. ≈ 16M bytes.) If such an error takes place, another global (static) status code variable, `gb_trouble_code`, is set to a non–zero value such that if performing multiple allocations in a row, an error on any one of these may be detected by means of a conditional check on said status code.

A convenience macro, `gb_typed_alloc`, is provided to allocate the required amount for a given type, through a quick `sizeof` on said type and a multiplication by the requested amount of objects n of that type. The resulting value is passed as part of an allocation request to `gb_alloc()` representing the number of bytes. Because this last routine requires an `Area` to be passed as well, it both allocates the required heap memory on said area, and returns a pointer that is cast to a pointer to the type passed as part of the `gb_typed_alloc` macro.

The `gb_alloc()` routine, at its core, only really `calloc()`s 2^8 items, each made out of `n` bytes passed as a parameter (having applied the ceiling function to those `n` bytes such that they are a multiple of the platform's pointer size, `char *` back when the program was written instead of `void *`, but 8 bytes either way (LP or LLP models would do just fine, and the program likely predates the time when proposals over these two "ended" other memory models.)) The specific size of each of the elements to be allocated is not completely clear to me just yet; DEK computes $^{n}/_{m} + {}^{2m}/_{m} + {}^{m-1}/_{m}$, which should theoretically resolve to $\approx {}^{n-1}/_{m^2} + {}^{1}/_{m} + 3$. But this meaning of this is lost on me right now. Maybe it only serves as a range restriction as per the notice in the documentation, which comments on old–style C having a hard limit on the byte size passed as the first parameter to `calloc()` (I've not found any such warnings on current–day, BSD–derivative, manpages.)

If the allocation is sucessful, the returned region of heap memory goes through three main "manipulation" steps, in the order described here.

1. The address lying `n` bytes (post–ceiling function) forward from the returned `calloc`ation is cast into the element of an `Area` (the underlying `struct area_pointers *`.)
2. A separate, temporary `Area`, is dereferenced (making use of array–to–pointer decay to reach directly for the first element, again a `struct area_pointers *`) to have assigned to its `first` field the starting address of the original `calloc`ation, and to its `next` field the dereferenced `Area` that was originally passed to the `gb_alloc()` routine (ibid.)
3. This latter (original) `Area` is again dereferenced to have its single element pointer `struct area_pointers *` alias the dereferenced element pointer of the former (temporary) `Area`.

This screams cycling references. Barring the fact that I don't yet understand the size of `n` after having applied (what seems like) a ceiling function to make it a multiple of the platform's pointer size, this only really takes a bunch of memory from the free store, offsets it and gets a pointer equivalent to the one in an `Area`, and proceeds to assign to that pointer's `first` field the non–offsetted memory address, and to its `next` field the original `Area`'s underlying element, to end up assigning to this same element the temporary's element we've been operating on all along.

The final state of the original, parameterized, `Area` has its single pointer element hold `first` to the actual start of the `callocation`, and `next` to the itself (starting off as a `NULL` pointer and following up with the allocations prior to the one performed on a call to `gb_alloc()`.)

Tracing the behavior of the `gb_alloc()` routine across a second call would have the passed `Area` be already allocated with a reference to `NULL` in its only element's `next` field, and a pointer to this same element's own adress minus an offset equivalent to the allocation of the current block in its `first` field. Fast forward to the end of the memory allocation request routine, and once the `callocation` has yield a pointer, the heap memory towards which the pointer leads is cast into the area's underlying pointee element type (`struct area_pointers`,) prior to performing an assignment to the `first` field of the temporary `Area` within the routine corresponding to the new block, after which the `next` field will be a pointer to the previous `Area` passed to the function, so that this same memory area holds a pointer to the start of the new allocation in its signle element's pointee `first` field, and another pointer to the allocation we had before in its `next` field.

Technically speaking, this stops being a potential source of self–referential pointer cycles after the first call, because if the user of the library takes care of keeping track of the `Area` (which they should,) then *the* pointer element within it (considering it's a singleton array of `struct area_pointers *`,) should lead to the "previous" allocation through its `next` field. So this field of the underlying structure acts as more of an indicator to the previous allocation, than as an indicator of which memory area comes after it.

The only way this represents a win in terms of efficency is if `calloc` is trusted to return contiguous memory allocations from some source array on the free store. This scheme would then have each `Area` hold both its own allocated size request (if it doesn't surpass `0xFFFF00` $\approx$ 16 M bytes,) and a pointer to the start of the previously allocated `Area`, which if the calls to the C standard library functions work as assumed by DEK, should yield contiguous addresses after $255 \times n$ bytes.

This is not the case in modern systems, and has never been the case in both ANSI and post–ANSI C. The win in efficency is arguable. Having single–threaded arena–like behavior encapsulated in a linked list–like `Area` object is not any better than having an array of pointers to memory allocated on

the heap through any of the `*alloc()` routines in standard C. Assumming they are all just resource handles acting as a bridge between the default system allocator and the library user, it's not feasible to try replicating this same strategy in Rust (because even if it relies on initialized, non–null memory ranges, it's still type–unaligned memory that would require calls to `std::mem::transmute()` to force a bit–level pattern reinterpretation, which is no better than `reinterpret_cast<>()` in C++, and that's dangerous in and of itself.)

A potential implementation that would both (1) assure *contiguousness* in the allocated memory, as well as, (2) allow for modern–day heap allocations without linked list behavior would be to use memory mappings from the UNIX API. This would be limitting for non–UNIX API users, but could be both safely implemented with the `rustix` safe wrapper around these syscalls, and is also how `malloc()` is implemented starting at certain memory request sizes in glibc. More specifically, this would use private, anonymous mappings on the process' virtual memory address space, letting the kernel allocate memory wherever it deems safe, while keeping a very similar memory arrangement, in terms of a pointer leading to the next `mmap`ed memory region. A single resource handle would hold reign over a conservative `Vec` capacity–like `MaybeUninit` region of memory, and any requests through the corresponding library API woudl be forwarded to such handle, which would return a pointer to the address range with the specified amount of bytes in the request, but would fail if the references taking up space in such region have depleted the required memory to fulfill the latest request. This, though, presents three issues.

- The `mmap`ped memory region must be large enough while still being conservative on the available memory provided to the process (must account for OOM in Linux and most BSDs.) The nomicon likely can given clues in chapter 9, where the reference implementation for `std::vec::Vec` is given along with an explanation on correctly doing OBRM.
- The resource handle would be forced to keep a reference count of both the referees making each request, as well as of the memory regions that they themselves hold resources over. This is pretty much replicating garbage–collected behavior of GC languages, except the reference count would only go down if a corresponding call to the deallocation routine is performed by the requestee that initially made the resource allocation call. This can be modeled through the `Drop` trait and associated `drop()` function on whichever type expects memory to be returned from this additional abstraction layer.
- Each call to `mmap()` would not be guaranteed to return a contiguous memory region, especially not if the starting address is kernel–dependent. A possibility would be for the initial `mmap()` call to be the *only* call to this syscall that lets the kernel pick an arbitrary address, such that subsequent allocations rely on having a handle over both the starting address and the offset that it is bound

by, then performing overlapping `mmap()` calls with the first parameter of the syscall denoting the starting address of the new range, and the parameter denoting the mapped size being a bounds–checked range over the original call. This could still potentially require another call to `mmap()` to fetch another chunk of memory from the process' virtual adress space, but if fairly decent heuristics can be found to set an initial "good" size, it should not happen as often as one would expect.

To round up the explanation on the memory allocation routines, the function in charge of having space deallocated, `gb_free()`, simply calls on a loop the C standard library function `free()` while first keeping track of a the corresponding `next` field in a separate pointer, such that the next iteartion of the loop may have another handle to free resources from. Of course, the exit condition is the `NULL`–ness the temporary pointer repeatedly fetched from the `next`.

The docs on the `Graph` type explain the existence of an assortment of routines to both create a graph, and attempt to efficently handle both vertices and edges. Most of the strategies followed by DEK are completely useless nowadays, and would require the use of platform–dependent code to rely on pointer arithmetic behavior that is not predictable outside MIPS and x86-32 and x86_64. A consequence is that the type system will be the only thing ported over to Rust, as the memory allocation practices are, in general, not apt for a portable, non–UNIX dependent program.

The type at hand considers 5 fields of chief importance for the graph, two `Areas` for data on arcs, strings and other auxiliary information; And 6 `util` union types as well as an additional field denoting a string whose single character sequences provide meaning to each of the utility fields.

The first five fields include a heap–allocated array of `Vertex` type. DEK uses heap allocations here for obvious reasons when it comes to vertex generation at runtime. There's two fields denoting the properties often found in textbooks by the same name, namely n, m, to denote both the number of vertices as well as the number of arcs in the graph (the predominant use of the arc terminology seems to be related to the fact DEK, and likely other sources, refer to these vertex links as "edges" only when speaking in the context of undirected graphs, but as "arcs" with both directed and undirected graphs.) Beyond this, each `Graph` is also outfit with two `Areas`, the first one used as a resource handle over both `Arcs` referred to by pointers in the vertices of the graph, and over strings with which to describe those arcs. The second `Area` is used as an auxiliary region for algorithms that may require scoped allocations bound within the graph instance, though some of the routines also use these, mostly for "tricks", as per DEK's own words.

The `util` unions on the graph are very much akin to those found in the `Vertex` structures except that they are also accompanied (within the `Graph` structure) with a single character array field acting as the discriminant of the union. This is truly a showcase of the severe limitations and safety issues

with C–style unions, and likely also the reason why the Rust rewrite should attempt to avoid such an approach. As a consequence, even though we speak of a string field, this is in actuality a purely character–based array where each character denotes in uppercase letters the purpose of the utility field at any given time (thus this acts as a (possible) warning to library users against the use of such fields without proper modification of the character discriminant, if the graph is to be used along with the exporting facilities the kernel routines provide for interaction with outside applications/libraries.) The characters in question follow the same semantics as the names in the union declaration itself, such that for each of the possible fields, it considers the complete set of union fields, and additionally a character `Z` to indicate that the field in question is not being used.

Each graph also holds an `id` field for the purposes of interaction with other graphs in the generative routnies of the GraphBase program itself. The tone with which DEK speaks while explaninig each of the elements of the API is very much that of a library providing graph primitives and not that of a library providing a graph testing framework, often mentioning the use of algorithms and how would these benefit from the fields provided to the developer of such routines, so it's as a conservative heuristic, I belive this kernel file should **not** be taken to be a realiable source to base the trait–based interface off of for the composable engine API covering the funtionaltiy of said core routines.

The reasoning behind the field describing the `util` unions present on each graph is that of providing an explanation for not only the graph's own union fields, but also that of providing meaning for the fields on each `Vertex` and `Arc` in the `Graph`. The selected formatting follows the afore–mentioned semantics, while expecting the user of such fields to interpret the first six indices as being those relative to the `Vertex` fields in the corresponding array, the next two fields as being those corresponding with the union fields present in `Arc`s, and the last six as being those in the `Graph` proper. The array, by default, is outfit with 15 characters, not so much because it requires of another field, but because it's implicitly also an ASCII–Z terinated string (a null–terminated string.) This design choice also implies the author is not expecting the API to be used without complete uniformity over the use of each `util` field for all vertices and arcs in a given graph (i.e. if the discriminant for some field of a vertex denotes a certain purpose for said vertex, such purpose is extended to any and all vertices in the graph.)

Because these fields' main purpose is that of providing a discriminant for the union and because DEK expects their use to be most often found in the I/O routines, they could be completely replaced with a trait–based implementation on the formatter API such that exporting was modeled after the Serde serialization/deserialization practices, which could make for an API that would be as extensible as the user's would require. A possible implementation would go through using a Serde–like custom `derive`

macro that would allow arbitrary user input on an external proc macro to set up the serialization and/or deserialization of the graph primitives involved in the generative routines of GraphBase.

The graph creation routine, `gb_new_graph()`, performs two main operations: (1) allocating space for the graph and a parameterized amount of edges *n* passed as part of the routine, plus an additional amount of vertices due to some algorithms requiring so, and, (2) setting up the value of a few file statics that cache part of the state of a graph upon creation to apparently make more efficient the use of certain routines often called right after the creation of a graph.

During initialization, the above routine will also set the `util` union fields to hold the discriminant variant standing for no information, namely character `Z`. This function will also set up the associated string identifier of the graph (the so–called `id`) to non–UID that is meant to be either immediately changed through one of two other routines, or left as part of the auxiliary graph IDs to be used in these two latter routines.

The functions concerning themselves with graph ID–setting provide either a single graph–to–graph way to perform such ID–setting operation, or alternatively a 2–graph to single graph ID setting routine to set a graph's ID from the ID of two other graphs and some additional strings. These routines will, for now, not be documented as a better approach would be the implementation of a UID algorithm for the graphs (which should be fairly simple, considering the `id` field serves as a means of inter–graph communication in the generative routines, and thus has no significance at the cryptographic level.)

As previously mentioned on the graph creation routine, one of the two main tasks it performs is to set up some external globals to cache graph state, and make sure fallible routines can indicate failure beyond the global error status codes (both the one set with the allocation request routines and the one found in the routines concerned with random number generation.) The set variables include symmetric equivalents for (1) arc allocation, as `Arc`s are allocated in bulk with `Area`s in `gb_alloc()` and a pointer is required to indicate both the availability of an arc that has not yet been added to the graph on a logical level and the possible failure state that should be compared with its symmetric counterpart, (2) string allocation, ibid. considering the use of an `Area` specifically for the purposes of storing both this and the prior resource, and, (3) graph allocation, ibid.

The routines involved with arc/edge creation are themselves wrappers for a more primitive routine that uses the current graph's main `Area` (recall the existence of two `Area`s, one of which was used for auxiliary purposes) to allocate a conservative default of 102 new arcs, irrespective of the amount actually required at any given point throughout program execution. There's two wrappers because one covers the usecase of having single–directionall arcs for directed graphs, while the other considers bidirectional arcs to denote the existence of an edge in an undirected graph (more generally, if a graph

is undirected, it is also a multigraph where any two vertices given by (i, j), always have two arcs $i \to j, j \to i$ of same weight, which voids direction of any meaning, and thus makes the graph *undirected*.)

Note this routine also uses the previously mentioned file statics to indicate either that the next available arc is the one offset by one byte after the address of the returned `gb_allocated` amount of arcs (thus one past the address denoted by the `first` field of the pointee of the element of the main `Area` of a `Graph`,) or that the allocation failed, and the "bad arc" variable must be assumed to be pointing past all elements that *should* have been returned from the `gb_tpyed_alloc` macro invocation.

The core logic of the routine, though, is fairly simple. If the global pointer holding the address of the next available (but not yet part of the graph) arc indicates it has reached the global holding the address found one element past the end of the `gb_allocated` `Area` currently tracked by the graph's main `Area`, then it's time to call `gb_alloc()` again and request another 102 arcs in a single block of memory that will now make the current graph's main `Area` become the holder of such resources, while keeping the previously allocated `Area` (used up) in its single element's pointee's `next` field (yes, the *previous* memory area is held in the `next` field.) Otherwise, it simply advances the global pointer tracking the next available arc in the current `Area` by 1 and returns another pointer aliasing that same pointer prior to having advanced it (thus the returned pointer is the one that was available on entry, while on exit, the tracking global pointer is past one element from the one returned and indicates again the next available but not set arc.)

The invariant held over which one is the "current" graph, and thus which `Area` should be the one being manipulated is weak in its semantics. Among the globals participating in the above routine, there's one that also tracks the graph that was created last with `gb_new_graph()`. That's the one holding all the responsibility with respect to which graph's `Area` is to be manipulated, pointed to, invalidated or extended. In this routine, a pointer is set to alias the returned graph. This has some severe limitations, that are only partially (and wrongly) bypassed by using the `gb_switch_graph()` routine to temporarily use some of the union fields in the "current" graph to store the information of such graph, and reset all union fields of the graph that is now meant to become the "current" graph. Then one may operate with the global referring to the "current" graph knowing as well that this refers to the graph passed as a parameter to the graph switching routine.

This procedure forces the "current" (prior) graph's union fields to be invalidated, and thus expects the user to only use those fields after any logic involving arcs and edges is completed. Otherwise, the union fields themselves will not restore their previous state. The most noticeable limitation, though, is the requirement on the passed graph having to have already gone through another graph switching routine itself to be "switched out." Because initially no graph could possibly have been switched out, DEK offers as an alternative to pass `NULL` to this routine right after the creation of a graph that is

“planned to be switched out”, only for the side effects it has on the global, which are the ones that force the requirement of the “current” graph (denoted by the corresponding global) having had to be switched in the first place.

This is going to need severe refactoring in Rust.

The wrapper to add an arc to a directed graph performs no logic beyond changing pointees of graph, vertex, and arc pointers, such that the new state of the adjacency list with an additional edge remains senseful. The wrapper around edge creation in undirected graphs is slightly more complex, but its implementation design will likely have to be trashed in Rust, as it heavily relies on the numerical value of pointers by directly comparing their adresses to determine a set of invariants over whether an arc can be found nearest in memory to its inverse arc (where the mapping of inverse arcs is that in which both such arcs, the *un*inverted and the inverted, make up the concept of edge, devoid of direction.) This also forces further restrictions on the call chain of arc creation routines as apparently mixing up this one with the other wrapper or with the underlying primitive function will lead to possibly undefined behavior or otherwise an ill–formed program.

No further comments will be made on these routines because they only perform trivial (and non–C standard conformant) pointer arithmetic that will be completely replaced in Rust.

The last routine the documentation comments on is concerned with string allocation for the purposes of vertex/arc labeling, making up the other “client” of the memory served by the main `Area` in a `Graph`. This function follows the same trend as the ones last commented on, using a few lines of pointer arithmetic to advance the pointer to the first character in a character array, and avoids using library functions for appending strings (`strcat` being the only one available back when this was written,) as they can be fairly inefficient. The approach to either returning the next available string or otherwise trying to allocate memory equivalent to that of the length of the string is used in the same way as with the arc allocation routines (except `gb_alloc()` is called directly with the requested size of the string because DEK likely assumed that either only the ASCII character set would be used, or otherwise the user would be prone to manually compute the length of any of their (1) wide strings, or, (2) UTF–8 encoded strings.)

If the corresonding global pointing to the next piece of memory in the `Area` of the “current” graph (itself denoted by another risky global) does **not** have the same address as the *other* global pointing one past the end of the valid, allocated `Area` and there’s enough space available in that same `Area` for the length of the string (i.e. offsetting the former global does not yet yield the latter global,) then the routine deems it safe to perform straight pointer arithmetic on the buffer starting at the in–bounds (“good”) global to yield as many bytes as the requested length indicates. Otherwise, it attempts to

allocate either the size of the string if this surpasses the default size request, or otherwise the default size request. Calls with a size smaller than the minimum in bytes are clamped to that minimum due to the fact requests to the allocation mediator routine provided by the library advise against using sizes smaller than 1000 bytes, as starting from that size the amount of syscalls required to actually reserve such heap memory, if available, should decrease. I highly doubt this is the case anymore nowadays, and DEK himself doesn't provide any reference to implementations of these C library functions that would lead one to believe this has changed.

This whole routine is completely out of the Rust rewrite. Maybe refactoring the standard Rust `String` struct to rid it of unnecessary behavior would be an option, but even then, the `std::string` module knows how to be efficient. And the most of this whole memory allocation strategy that's getting into the refactor is (maybe) an abstraction layer over the `std::allocator` structs to further mediate between the library user and the system allocator (only if that's possible without forcing it on the dependent crate.)

There's also a routine to free the resources of the graph, that calls the corresponding memory–freeing routines on `Area`s, and `free()` on the passed graph (`Area`s are not used to keep a record of `Graph`s in `gb_graph.w` but that may very well be the case in the generative routines, considering DEK exposes memory areas explicitly as part of the public interface of the library kernel modules.) This is not going to be discussed further, because it's completely useless with RAII in Rust. The only possible modification to the `drop()` trait method that would be required to implement similar semantics would require overwriting the `std::allocator` structs, which may or may not be possible if the dependent crate is also affected by such changes to the system allocation Rust API. The `std::alloc` module does mention that the attribute `[global_allocator]` can only be used once on a crate, but doesn't further specify whether dependencies of that crate will be forced into using the same memory allocator. It does mention, though, that recursive dependencies of a crate (so I'm assumming this includes both explicit cargo dependencies and whichever dependencies these themselves include) can only ever specify this attribute once. This does mean that if this library (GraphBase) includes code that overwrites the allocator, (direclty or indirectly) dependent crates will be forced into using the same allocator, as that's what I can infer from the fact that a no two crates involved in a package can have the attribute used more than once.

Initially, I believe it best to implement the library in terms of `std` defaults, and worry later about how could the DEK memory management strategy be implemented in Rust.

The docs now move on to the part of the library implementing functionality for fast O(1) vertex lookup through their string labels. This is considered in the context of hashing with separate chaining, using the derived results from DEK's conclusions on the number of probes that would be required

to compute the number of comparisons between the (hashed) input key and some search key that would change as the symbol table was traversed. In the initial implementation, I believe the Rust code should just use the algorithms in the standard library to compute the hashes of the string keys, and store them in either one of an auxiliary hashmap stored within the overarching graph representation/ DS, or otherwise use an extension interface to allow the users to make arbitrary use of the hashmap DS as they see fit. A possible implementation for this would be the use of proc–macros for compile–time addition of a field in the graph DS, such that use of that feature is gated to a user request on codegen.

Another reason to get the proposed strategy replaced is due to the fact the number of nodes after creation of the hashmap ought be kept consistent by the user, as otherwise the whole hashmap would have to be rehashed. I'm not completely sure about the extent of this limitation in Rust's `HashMap`, but at least the user is assured that if such thing were required, it's quite possible that the container could make do by itself without manual intervention. There's also the fact that auxiliary global pointers are messed with, which is not exaclty ideal. Add to that the use of `util` unions on the `Graph`, and we're pretty much binding the user to another set of strict requirements on the extent of their use of such fields; Especially considering the note warning against possible system crashes if one does not meddle with great care.

In Rust, I think both the union fields and the additional hashmap could be implemented in terms of proc–macros. These would allow the crate user to generate, on a case–by–case basis, instances of the `Graph` DS proposed by DEK with well–defined extensions to the `structs` through attribute–like macros. The feasibility of this, though, is quite uncertain. On the one hand, tagging existing structures with attribute–like proc–macros seems like an option worth considering. On the other hand, one soon realizes that such need would not arise in a dependent crate unless the user designed themselves the graph DS. Then, if one considers providing pre–existing data types for graphs, the attribute–like macros would be forced to either act as extensions to all generated instances of the graph DS, or otherwise… God knows. The ideal behavior, just to get the idea out, would be for the user to create the graph DS and then allow them to, on a case–by–case basis, tag some given instance with the attribute. Such thing would allow the underlying proc–macro to create a new type for the graph (with a possibly mangled name that would include an custom identifier; Maybe a hash key relating to the identifier of the variable with the attribute?) such that this type could be used for the purposes of plugging a given struct into a generic interface expecting a type that implements some trait that the newly generated structure could also implement. Whether this is at all possible in any context in which a user is required of a type implementing a given trait for a specific function, is very much related to the extent with which the Rust grammar allows flexibility in the declaration of new types and the scopes in which this can happen.

The proc–macros idea seems feasible. The only thing to consider as a notable difference from what I commented on above is the way one would encode information relative to a newly generated type. Simply using a hash function is not enough, as that forces holding the invariant that provided two equal input keys, the same hash is returned. Of course, if we consider variable shadowing, this stops providing the proc–macro with unique IDs. The solution would be to either (1) consider a UID generator (through an external library or through a manual implementation of one such algorithm,) or, (2) consider hashing in terms of the `Span` associated with some token in the `TokenStream` passed to the underlying proc–macro function signature.

This seems very much possible. Apparently, only *items* in the Rust grammar can be considered as being capable of having an outer attribute applied to them, but it turns out statements, including `let` statements, may also have outer attributes applied to them. The extent with which some such outer attribute may have an effect on a statement, as per the reference's concepts of both, is not yet clear to me, though. The reference speaks of "applicability" when referring to the possibility of having outer attributes be used on a statement, including a `let` statement, but such "applicability" is not further developed in the context of whether it is restricting in nature or if it only offers an example of the uses it currently finds in the `rustc` compiler.

In the case that `let` statements also applied here, then the afore mentioned idea could very well be feasible. Regardless of whether the type inference algorithm was run prior to forwarding the `TokenStream` to the proc–macro, it would still be possible for the proc–macro to attempt its own form of inference on the raw syntax.

The proc–macro implementation, though, would be best served by an outer attribute that applied to some tuple–like unit struct such that the user decided on the token identifier and scope of the type about to be generated. Because the attribute would likely have to be the same for the purpose of accessibility to the user if the API ever extended beyond this basic functionality, this should prove to be enough. Because outer attributes (as attribute–like proc–macros are, when parsed post–tokenization) are expected to completely consume the `TokenStream` of the second parameter to the macro's underlying function signature, taking in the user type and generating all of the fields, as well as the required trait implementations for some generic function expecting a type implementing a certain trait, should make this whole idea possible.

Of course, further refinements need to be made to actually come up with a macro that doesn't try to cover too much ground. This would likely mean considering whether it's idiomatic to expect the user to have some trait automatically implemented from simply invoking an attribute–like macro, and not from having a traditional derive macro implemented on the resulting type.

I'm going to leave this in the backburner while I continue inspecting the GraphBase codebase.

1.1.3 I/O routines (`gb_io.w`)

This file contains all of the logic concerning the processing of input data files within GraphBase in case a user is in possesion of some file exported through the (non–kernel module) `gb_save.w`. This, indeed, implies that the file does not exist as a set of both input *and* output routines, but rather as a set of routines to be used exclusively for both checking the contents of a file and getting its contents parsed into structures that GraphBase understands (though I'm not so sure on the latter.)

These routines seem to provide functionality mostly specific to either the part of the GraphBase not implementing functionality directly interfacing with the core logic of the program, or otherwise interfacing with both input and output in the `gb_save.w` module. The only thing requiring a reimplementation in Rust is going to be the backwards–compatible interface to allow reading in files through the same "protocol"/"language" as the one DEK uses in the origina GraphBase. Even though I plan on allowing the user to automatically derive the internal types used for the whole codebase with an arbitrary serialization format through the `serde` crate, it's very much a necessity to have all `.dat` files still work with the rewrite.

This would force the implementation of at least part of the non–parsing–specific routines in the core module. Things like the universal character set that DEK uses instead of restricting the symbols on the data files to either one of (1) ASCII, or, (2) ECBDIC (or possibly another, completely different, post–90s character set encoding) could be replaced simply with Rust's native UTF–8 strings. Other elements of the interface explictily exposed to the library user should straight up be removed, considering a large part of the prep work prior to parsing the data files is tied to the limitations of C and computers as a whole back when the GraphBase was originally written.

The system–independent encoding that DEK uses is not completely system–independent, because the setup routine of the `icode` array holding the numerical values to which some character maps is performed through a function that expects the user's machine to evaluate the numerical value of each one of the characters that the `imap` string (really only a string out of convenience, as each single–byte character is layed out as if they were contiguous elements in an array) contains. Such a value is assured to change in each non–ASCII or UTF–8–compliant system, and thus the actual offset added to the start of the `icode` array while filling it with the accompanying numerical value is different depending on the C runtime's evaluation of the character, which is most likely also tied to the character set encoding of the system in which the program is being executed.

Because current–day limitations on character encodings do not radiate from this type of issues, I believe it best compatibility is only upkept with the parsing logic.

After having read all of the docs on the parsing logic, it should be fairly simple to rewrite. The only parts of a `.dat` file that GraphBase expects to be conformant with a specific grammar are the first four lines and the last line of the file. Every line of a `.dat` file should contain at most 79 characters, out of which the first line needs to contain the first few characters as a substring matching the following regex.

```
^\* File "([^"]+)".+
```

The second and third lines of the file are only matched against the same `*` character, and can thus and often will (at least in the GrahpBase repo data files) contain some description of the source/purpose of the data set, as well some licensing on how would the author like others to distribute the file.

The fourth line of the file will include information relative to the checksum, which allows performing a fast computation of the "expected" contents of the input string and thus lets the program bail out as it is reading the buffer if the expected input does not adjust to the result of the checksum formula. The details of the checksum used by DEK will be commented on later. The specific format of the line should adjust to the following regex.

```
^\* \(Checksum parameters (\d+),(\d+)\)$
```

The first capture group in the regex corresponds with the expected number of lines after the checksum (fourth) line, and *not* counting the last line (the one with no data, but special formatting.) The second capture group corresponds with the final "magic number" that DEK uses to compute the checksum from the entire contents of the file, through the formula we'll discuss after we're done with the grammar describing file formatting.

After this line, the parsing routine simply sets up the required (global, and thus inherently unsafe) buffers for other routines to manually continue the parsing process on the rest of the file contents, making sure that there are exactly as many lines as the first checksum parameter indicated at the start of the file.

Once the specified number of lines has been read through other routines (that are quite possibly going to get replaced in Rust,) another parser routine is expected to be called to check for the contents of the last line in the file to conform with the following regex.

```
^\* End of file "([^"]+)"$
```

We move now onto the checksum formula that DEK uses to check that the contents of the file match with the second parameter of the fourth line, itself corresponding with the resulting magic file that the library routines should have gathered from computing the (series–like) formula.

The checksum is computed in terms of the formula

$$\left(\sum_{l} 2^{l} \cdot c_{l}\right) \bmod p, \tag{1.2}$$

where p is a large prime, and the values of c_l depend on DEK's character set enconding.

Each possible value of c_l corresponds with a numerical value that maps 96 admitted characters into a symbol table that hashes them into the range `0..=96`. The checksum is then computed by reading the characters from each lines of the file and getting, on a character–by–character basis, the hashed numerical value added to some initial value a, itself starting as the old value of the checksum or as 0 when reading in the first line of input, in the following loop formula.

$$a = (2 \cdot a + c_l) \bmod p. \tag{1.3}$$

This should hold so long as the string yields a non–null character, such that the value of the checksum is only considered to be valid if the line adjusts to the set length of 79 characters. Each "old checksum" is then added to the value of the newly computed temporary (upon hitting end of line by hitting the null terminator of the passed string) for the routine to return the new value of the checksum as a function of both. This is supposed to evaluate to Equation (1.2) once the entirety of the file has been read (where we define *entirety* as all lines post the checksum–parameterized (fourth) line, and prior to the last `*`–prefixed line, indicating the end of the input data set with the same name as indicated in the first `*`–prefixed line, for a GraphBase–conforming data set.) The result of the last computed checksum is the one that is then compared with the second parameter of the fourth line of the `.dat` file.

In terms of the grammar of expected data, the GraphBase makes no specfication beyond providing library user with three routines to either (1) parse a string until meeting some other passed delimitter, (2) parse a digit in some given radix d, by checking through the `icode` array the numerical value of the read number; This is possible because all character–encoded numbers are meant to map to the same numerical values in DEK's encoding, or, (3) read in a whole number by performing an operation akin to that of reading a single digit, but instead repeating in a loop and adding up the values to some temporary k that is returned with the correct powers of the passed radix for each digit of the processed number. This implies that the actual data between the first four lines of the file and the last line is only expected to comply with the conditions on line length (79 characters, not accounting for newline termination,) and on character set encoding (96 characters including the standard 94 visual ASCII characters, the `\n` escape sequence and the whitespace separator.)

The notes on the routines for I/O should be mostly done now. The rest of the work left on the kernel routines concerns itself only with the sorting module, and actually understanding the random number generator now that I have possession of volumes 2 and 3 of DEK's magnum opus.

1.1.4 Sorting routines (`gb_sort.w`)

This module holds the routines and types used to perform linked list sorting of any type involved in the GraphBase graph primitives (so any type among `Graph`, `Vertex` or `Arc`.) The nature of these subroutines is not one where sorting is performed along with some other type–specific operation, but rather one where the elements being sorted are themselves abstracted as pointers to `nodes` such that sorting is done independent of both (1) the pointee's type, and, (2) independent of the actual type used for these `nodes`.

The reason then for using a specific type (`node`) owes to the fact the sorting algorithm requires of the properties of a linked structure, whereby one instance of such a *node* is always bound to yield the "next" element in a *list* collecting all elements under consideration for sorting purposes. But the overarching container with pointers to each of these elements isn't required to hold a specific type of pointer; instead, it considers pointers (of pointee `char` type, because `void` wasn't legal C when GraphBase was written) to implicitly denote to the library user that for the (sorting) routines to work correctly the minimum "interface" for the sorted–through elements is supposed to have as its first two fields the same two fields in the `node` type, these being the only ones used in the GraphBase sorting routines.

In terms of the actual sorting algorithm, it's radix sort with radix 256, and 6 digits under consideration. The details of this algorithm in conjunction with the linear congruential engine used for random number generation (under `gb_rand.w`, and commented on in Section 1.1.1) are still something I'm not completely confident I understand.

I will proceed first with the logic involved in the form of radix sort used by DEK. The algorithm, as explained in [6], considers a set of records n that is equal to the total amount of numbers to be sorted over. This abstraction is necessary because each of those records keeps both a `key` field, as well as a `link` field, such that in actuality it is a form of (primitve, singly) linked list that, initially, has every single element's `link` field pointing to `NULL`.

This abstraction is provided by the example `node` structure, itself having the exact same `key` and `link` fields as the ones exemplifies in [6]. Then, it keeps track of a collection of as many queues as the numerical value of the radix for each of those digits is. Following, for as many iterations as the number of digits in the largest of the `node`'s `keys` under consideration, the algorithm will consider the iteration number as the queue to operate on, and subsequently proceed to add the `nodes` whose (`key`) digit at the position currently under consideration (the iteration number) is equal to the iteration number. This introduces the second constraint imposed by these sorting routines; The structure in use (in place of the example `node`) must additionally employ 32–bit unsigned integers in its `key` field

(though this is also due to the range of the random number generator used for the first two passes of radix sort.)

It will do this consistently for each of the records (the `node`'s numerical `keys`,) and then it will call another subroutine ([6, Alg. 5.2.5H]) that will link the top elements of each queue with the bottom element of the queue following that one queue, where the concept of order between queues is upkept thanks to the contiguous collection within which the queues themselves are contained. This step allows the algorithm to repeat the above steps (the steps explained in the previous paragraph) on the next least significant digit of the numbers (so starting from the least significant digit, it moves to the right,) producing a completely different order, but this time not considering the records in the provided order, but rather in the order that they were left on (recall they are really nodes in a linked list) after performing the above subroutine to connect the top record of each queue with the bottom record of the next queue.

Repeating this for as many digits as the largest `key` in the collection being sorted has, allows a "progressive" form of implementation, bounded in the `gb_sort.w` module at 6 passes.

From looking into the implementation of radix sort in [7], it seems the above explanation applies to the method followed by DEK in [6], but it isn't necessarily unique nor does it strike me as any better than the proposed approach in the former reference. T. H. Cormen, C. E. Leiserson, R. R. Rivest, and C. Stein algorithm simply considers each of the digits of the multiset of numbers in the input collection, and proceeds to apply some other stable sorting algorithm only on the digit under consideration in the current iteration. This keeps repeating itself for as many times as there are digits in the largest number in the collection, padding with left zeroes numbers with a smaller amount of digits.

The algorithm of choice as per [7] for stable sorting each digit is *counting sort*. Following, I provide a summary on its workings, based off of the explanation given by T. H. Cormen, C. E. Leiserson, R. R. Rivest, and C. Stein.

The routine involved here is not in–place so there are, at least in theory, two more memory allocations performed on each call to counting sort. The initially allocated array has as many elements as the numerical upper bound on the input array, plus 1; That is to say, it allocates memory for as many elements, plus 1, as the largest contained number in the array to be sorted. This procedure is thus best used when it is known in advance that the largest number contained in the input array is the number of elements in the array or ever so slightly larger than it.

After this, it performs an initial, linear cost pass over the input array to consider the amount of times the index of each element in the newly–allocated array repeats itself in the input array to be sorted. The

new array thus serves the purpose of a frequency table akin to that used when building and operating with a Fenwick tree (also known as a Binary Indexed tree,) such that the index serves as indication of the element in the input array, and the actual element in the frequency table provides the amount of times that one number is seen (repeated) in the source collection. Then the new array is traversed again in another linear cost operation (this time dependent on the largest value contained in the input array,) and proceeds to update the frequencies of each element by adding its current frequency to the frequency of the element that came before it (even though the assymptotic cost is linear in nature, the computation involved starts at the second element, because the first element has no peer preceding it and thus has the "right" frequency assigned to it during construction of the frequency table.) Once this is done, the new array contains, at each index (recall the indices indicated the actual values contained in the input array) the amount of elements that are smaller than it (an implicit consequence is that even for indices referring to numbers not appearing in the input array, if some *smaller* number *did* appear in the array, they would have its frequency added to it.) This information already encodes the ordering of the element with the same index as the element with the same value as the index in the frequency table, and so the only thing left is to traverse the length of the original array, and index each of the elements of the frequency table with the yielded values of the input array, to get the position in which one must put the element indexed at the original input array into another output array (this output array is the second allocation that makes this algorithm non–in–place.)

Because I've yet to discuss the workings of the specific implementation that DEK uses for the GraphBase sorting module, I've not yet decided on whether I should use the approach indicated in [6], or otherwise follow through with the above approach. In terms of DS layout, it definitely seems like the former is better, but this is only a "gut feeling."

From rereading the initial documentation comments on the module, it seems the whole purpose of this sorting routine is not to perform a stable sort, but rather to purposefully shuffle elements that compare equal in random ways (i.e. to perform an intentional unstable sort by randomly laying out elements with a equal partial ordering relation,) while still ending the routine with an increasing sequence of elements layed out in the same linked list as the one exposed in [6].

For that, DEK expects the users of the library functions to provide a structure that aligns with the requirements of the example structure `node` (that we already spoke of when commenting [6, Alg. 5.2.5R].) Because of `struct` layout constraints in C compilers (except when using `#pragma`s to change field packing behavior,) the fields used in the sorting routine (`gb_listsort()`,) namely `key` and `link`, ought be the first two fields in the structures that the users provide to the library function. This limitation could be easily circumvented through codegen in Rust, and it would likely not affect compile–times. This is feasible because DEK expects this sorting algorithm to be used selectively

by certain algorithms that require adscribing order (or possible disorder if all `keys`' partial order compares equal) on some input data set, like the words contained in the `words.dat` file; thus, the macro itself would be used on a case–by–case, (hopefully) fine–grained basis by library users. This may not be possible, based on the functions provided in the standard library for sorting elements in slices (`sort*()` and `sort_unstable*()`.) The variants of these functions that accept a closure expect to have an `Ordering` returned, which means that there must be at least a binary weak ordering relation between elements. If I try to manipulate the output based on a precomputed result of the `partial_cmp()` function, I would run into the issue of having to force a non–equal result between elements that, `key`–wise, would compare equal. Fast forward to the end of the sorting algorithm, and I would end up with a tangled mess of unordered `keys`, that have taken into account satellite data ordering as its primary resolution strategy, instead of `key` ordering.

To this extent, this single routine may be replaced with a regular, contiguous heap–memory allocated container like Rust's `Vec`, and instead of performing radix sort, performing a regular stable sort with control over how does the comparison function resolves through a closure. This should allow the closure within the stable sort to perform the same random shuffling of values that evaluate to partial equality, and for any other value, resolve to the built–in total ordering of integral values.

The first part of the routine performing the initialization step **R1** in [6, Alg. 5.2.5R], may be potentially unsafe if the next returned random value covers the full unsigned integer range, namely 2^{31}, because 23 right–shifting operations are not enough to explicitly cover the range 0–255, which is the only valid range for the array that is being indexed with the returned (and post–bit shifted value of the linear congruential random number engine.) The range covered is $[0, 256]$, but it ougth be $[0, 256)$ (note it's closed on the end range of the segment) to index the array without an out–of–bounds miss.

The last two passes of the algorithm are based on the MSD–pass idea proposed at the end of the [6, Sec. 5.2.5]. They follow the same principle as the one used for LSD passes. All passes also base their behavior off of the assumption that the keys will only ever hold some number with 6 digits tops, as the range of values for p is $[0, 6]$ (where p here has the same meaning as the one adscribed by .)

This module will likely also be completely rewritten, as the only thing that it attempts to do efficently is to perform stable sorting with the effects of unstable sorting, making sure such effects are as non–deterministic in nature as possible. To that extent, it uses radix sort with the same linked list–like, and queue–like behavior as proposed in [6], but on the first two iterations of p considers randomly shuffled keys, which are then reordered into the desired layout. The goal is to compute the partial order of nodes in terms of their `key` fields to produce an increasing sequence in a contiguous container, while randomly shuffling values that compare equal. To allow further flexibility to library users, the module should be refactored into using a trait–based implementation with the same codegen idea as

proposed with the `util` unions (commented at the end of Section 1.1.2) in the graph primitives. This should allow deriving the `PartialOrd` trait such that the satellite data remaining on the graph type determines the final ordering of the elements.

This should about do it with the `gb_sort.w` module. Before moving on to the generative routines, I believe it best to revisit the `gb_rand.w` module, as I didn't have the bibliographic references the author used to implement the linear congruential engine when I commented on the codebase.

1.1.5 Random number generation revisitted (`gb_flip.w`)

On the discussion about the potential unsafety of the sorting routines when calling the random number generator routine `gb_next_rand()`, the `gb_flip.w` module does mention that the range of returned numbers is bound to that of *signed*, and not *unsigned* integral values; Contrary to what I said, which included the range 2^{31}, the real range ends at $2^{31} - 1$, which does mean the eight most significant bits extracted on the first two runs of the partioning scheme are just fine, as the maximum value they map to is `0xFF`, which when shifted right 23 bits, should yield a number in the expected range $[0, 256)$.

[6, Sec. 3.6] refers to older but analogous content to the one included in S. Skiena's catalogue under [1, Sec. 16.3]. The presented results are subpar compared with those presented by the more modern linear congruential engine formulas and heuristics discussed by the latter. Chief among the deficiencies of DEK's methods is the period of the engine; 2^{55} on runs that don't hit the limitations commented on both the module file and Section 1.1.1 concerning the birthday spacings test. In contrast, S. Skiena presents multiple better solutions known today, like the *Mersenne Twister* engine with a far larger period ($2^{19937} - 1$) and alternative implementations depending on the machine's word length.

Additionally, the cycling computations that DEK advises for users of the library that require a certain result against the afore mentioned test, is not encouraged in [4] because apparently it doesn't solve other deffects related to the predictability of generated sequences.

The random number generator module is getting completely replaced with another trait–based interface requiring of whichever methods one may need to compute random numbers in any general application under consideration within GraphBase. For testing purposes during development, the `rand` crate will be used instead. Once I get everything working, I may consider revisitting the algorithm in the original GraphBase and providing it as a `ClassicBackend` for the random number generation API.

1.1.6 Sorting routines revisitted (`gb_sort.w`)

Future efforts should focus on better understanding the sorting module, as the method explained in [6, Sec. 5.2.5] is clear, but the feasibility of implementing it in conjunction with a random number generator is not something I completely understand just yet. The purpose of this second exploration

of the module should be evaluating whether it is feasible to provide the same functionality with a different implementation I understand better.

Upon further inspection of the documentation, it seems the module is not as limited as I initially believed it to be. It turns out the array `gb_sorted[]` exposed as part of the public interface is actually made up of linked lists, and not of the final records themselves as they are presented in [6, Alg. 5.2.5R, Alg. 5.2.5H]. Indeed, in the example code snippet given at the start of the module, the initial traversal over elements of the array is followed by a nested traversal through the nodes of each linked list, such that for each of the 127 linked DSs, a single pointer is stored to the starting node, that itself will hold a pointer to the next node in the corresponding list.

This would imply that if `gb_sorted[]` is to reflect the sorted order of all of the records (to use the terminology in [6, Sec. 5.2.5],) such that the linked list indexed at position 0 holds the first n smallest elements of the original linked list, then the initial partitioning is meant for both random shuffling and to separate all such elements in 256 linked lists, each an element of `alt_sorted[]`.

Another conclusion I've reached is that random shuffling is performed on a best–effort basis, as it's not enforced throughout sorting, but rather during the "initialization" (partitioning) steps. DEK then puts his hopes into the initial sequence being sufficently shuffled for the next 4 passes of radix sort to do their job just fine with respect to the `keys`, such that the satellite data of `keys` that compare equal is left in the same position as given post–random shuffling thanks to the stability of radix sort.

Two paragraphs above, the use of a general bound n for the number of elements in one of the linked lists in the sorted array of linked lists was not defined. To determine the index of the specific linked list in which a `key` or `key` range lies in, one may consider the following bounds on the value of the `key`(s) in relation with the valid range of indices into the array.

For index $0 \leq j < 256$, linked list `gb_sorted[`j`]` contains the range of `keys` $\left[j \cdot 2^{24} \leq \texttt{key} < (j+1) \cdot 2^{24}\right]$.

Bibliography

[1] S. Skiena, *The Algorithm Design Manual*, 3rd ed.

[2] D. E. Knuth, *The Stanford GraphBase*.

[3] D. E. Knuth, *Literate Programming*.

[4] *Seminumerical Algorithms*, 3rd ed., vol. 2. in The Art of Computer Programming, vol. 2.

[5] N. Deo, in *Handbook of Data Structures and Applications*.

[6] *Sorting and Searching*, 2nd ed., vol. 3. in The Art of Computer Programming, vol. 3.

[7] T. H. Cormen, C. E. Leiserson, R. R. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed.